

Solutions to Problems related to Decision Making Under Uncertainty & Reinforcement Learning

ELG 5218 – Uncertainty Evaluation in Engineering
Measurements and Machine Learning

Instructor for this lecture: Bingze Xia, University of Ottawa

April 2026

Contents

I	Chapter 34: Decision Making Under Uncertainty	2
1	PART A: Conceptual Questions	2
2	PART B: Mathematical Derivations	5
3	PART C: Computational Problems	8
4	PART D: Analysis & Comparison	10
II	Chapter 35: Reinforcement Learning	12
5	PART A: Conceptual Questions	12
6	PART B: Mathematical Derivations	15
7	PART C: Computational Problems	17
8	PART D: Analysis & Comparison	19
9	PART E: External Resource Problems	22

Part I

Chapter 34: Decision Making Under Uncertainty

1 PART A: Conceptual Questions

Problem A1: Frequentist vs. Bayesian Risk

Explain the key difference between frequentist risk and Bayesian posterior risk. In what sense does each approach treat the unknown state h and the data x ?

Solution A1

The two frameworks differ in what they treat as random:

Frequentist risk: The state of nature h is treated as a fixed (but unknown) constant, and the data X is treated as random. The risk of a decision rule δ is:

$$r(\delta | h) = \mathbb{E}_{p(x|h)}[\ell(h, \delta(x))].$$

This averages the loss over all possible datasets that could be generated given the true h . The problem is that h is unknown, so we cannot compute this directly. Solutions include minimax (worst-case over h) or integrating over a prior.

Bayesian posterior risk: The data x is treated as observed (fixed), and h is treated as a random variable with posterior $p(h|x)$. The posterior risk of action a is:

$$\rho_\pi(a | x) = \mathbb{E}_{p(h|x)}[\ell(h, a)].$$

This computes the expected loss for a specific action given the observed data.

Key insight: By Fubini's theorem, both paths lead to the same overall Bayes risk:

$$R_B(\delta) = \mathbb{E}_{p(x)}[\rho(\delta|x)] = \mathbb{E}_{p(h)}[r(\delta|h)].$$

Practically, the Bayesian approach tells you *how to construct* the optimal decision for a given observation, while the frequentist approach tells you how to *evaluate* a decision rule.

Problem A2: Loss Functions and Optimal Estimators

For each of the following loss functions, state the Bayes-optimal estimator and explain intuitively why it is optimal:

- (a) Zero-one loss: $\ell_{01}(h, a) = \mathbb{I}(h \neq a)$
- (b) Quadratic loss: $\ell_2(h, a) = (h - a)^2$
- (c) Absolute loss: $\ell_1(h, a) = |h - a|$

Solution A2

- (a) **Zero-one loss $\rightarrow$ MAP (posterior mode):**

$$\delta^*(x) = \operatorname{argmax}_{y \in \mathcal{Y}} p(y | x).$$

The posterior expected loss is $\rho(\hat{y}|x) = 1 - p(\hat{y}|x)$. Minimizing this is equivalent to maximizing $p(\hat{y}|x)$. *Intuition:* every mistake costs equally, so pick the single most probable class.

(b) **Quadratic loss $\rightarrow$ MMSE (posterior mean):**

$$\delta^*(x) = \mathbb{E}[h \mid x] = \int h p(h|x) dh.$$

Derivation: $\rho(y|x) = \mathbb{E}[(h - y)^2|x]$. Taking $\frac{\partial}{\partial y}\rho = -2\mathbb{E}[h|x] + 2y = 0$ gives $y^* = \mathbb{E}[h|x]$.

Intuition: large errors are penalized quadratically, so the estimate is pulled toward the “center of mass” of the posterior.

(c) **Absolute loss $\rightarrow$ posterior median:**

$$\delta^*(x) = \text{median of } p(h|x).$$

Intuition: all errors are penalized linearly regardless of direction, so the estimate splits the posterior probability mass equally. This is more robust to outliers (heavy tails) than the mean.

Problem A3: Bandits vs. MDPs

Explain the key structural difference between a contextual bandit and a Markov Decision Process (MDP). Why does this difference make MDPs fundamentally harder?

Solution A3

Contextual bandit: At each step t , a context (state) s_t is drawn independently of the agent’s previous actions. The agent picks action a_t , receives reward r_t , and the next context s_{t+1} is drawn from $p(s_{t+1}|s_{1:t})$ *independent of* a_t .

MDP: The next state depends on the current state *and* the agent’s action: $s_{t+1} \sim p(s'|s_t, a_t)$. Actions have long-term consequences—choosing action a now changes which states are visited in the future.

Why MDPs are harder:

- The agent must consider *long-term consequences*, not just immediate reward. A locally suboptimal action might lead to high-value future states.
- The exploration–exploitation tradeoff is compounded: exploring in one state changes the distribution of future states.
- Credit assignment is difficult: a reward received many steps later must be attributed to earlier actions.
- Solution methods require reasoning about discounted cumulative returns, value functions, and the Bellman equation—none of which arise in a simple bandit.

Problem A4: Value of Information

In the oil wildcatter example from the lecture, explain what the Value of Perfect Information (VPI) measures and how it is computed. Under what condition should the wildcatter perform the sounding test?

Solution A4

The VPI measures the maximum amount a rational agent should be willing to pay for access to additional information before making a decision.

Computation:

1. **Without test:** Compute $\text{MEU}_{\text{without}}$ by selecting the action with highest expected utility under the prior. From the lecture: $\text{EU}(d = 0) = 0$, $\text{EU}(d = 1) = 0.5(-70) + 0.3(50) + 0.2(200) = 20$, so $\text{MEU}_{\text{without}} = 20$.
2. **With test:** For each possible test outcome s , compute the posterior $p(o|s)$ via Bayes' rule, find the optimal decision, and weight by $p(s)$:

$$\text{MEU}_{\text{with}} = \sum_s p(s) \max_d \text{EU}(d|s) = 0.41(0) + 0.35(32.9) + 0.24(87.5) = 32.2.$$

3. VPI:

$$\text{VPI} = \text{MEU}_{\text{with}} - \text{MEU}_{\text{without}} = 32.2 - 20 = 12.2.$$

Decision rule: Perform the sounding test if and only if its cost $< \text{VPI} = 12.2$ (i.e., \$122k in the problem's units). VPI is always ≥ 0 because information can never hurt a rational decision-maker (you can always ignore it).

Problem A5: Discount Factor Interpretation

In an MDP with discount factor γ , explain:

- (a) What happens when $\gamma = 0$?
- (b) What happens when $\gamma = 1$ (for the 1D grid world from the lecture)?
- (c) Why is $\gamma = 0.9$ often a good practical choice?
- (d) Give the probabilistic interpretation of γ .

Solution A5

- (a) $\gamma = 0$ (**completely myopic**): $G_t = r_t$. The agent only cares about the immediate reward. In the 1D grid world, only $Q(S_3, \downarrow) = 1$ is nonzero because only the action immediately before the terminal reward matters.
- (b) $\gamma = 1$ (**no discounting**): All future rewards are weighted equally. In the 1D grid world, every state-action pair that eventually reaches the goal has $Q = 1$. This provides *no directional guidance*—the agent doesn't prefer shorter paths.
- (c) $\gamma = 0.9$ (**balanced**): The agent values future rewards but prefers them sooner. In the grid world, $Q(S_1, \downarrow) = 0.81 = 0.9^2$, reflecting that the goal is two steps away. This “discounting encourages speed”—the agent prefers the shortest path.
- (d) **Probabilistic interpretation:** $1 - \gamma$ can be interpreted as the probability that the interaction terminates at each step. Under this view, the discounted return equals the expected undiscounted return in an environment where episodes end randomly with probability $1 - \gamma$ per step.

Problem A6: From MDP to POMDP

A Partially Observable MDP (POMDP) extends the standard MDP framework.

- (a) What is the key structural difference between an MDP and a POMDP?
- (b) Define the *belief state* b_t and explain why it is necessary.
- (c) Give a concrete example of a POMDP from robotics or autonomous systems, identifying the true state, the observation, and the source of partial observability.

Solution A6

- (a) **Key difference:** In an MDP, the agent directly observes the true state s_t at every step. In a POMDP, the true state s_t is *hidden*; the agent instead receives a noisy observation $o_t \sim p(o \mid s_t, a_t)$ that provides only partial information about s_t .
- (b) **Belief state:** The belief state is a probability distribution over possible true states given the agent's history:

$$b_t = p(s_t \mid h_t), \quad h_t = (a_0, o_1, a_1, o_2, \dots, a_{t-1}, o_t).$$

It is necessary because the agent cannot act optimally based on a single observation (which is ambiguous). The belief state summarizes all relevant history into a sufficient statistic, converting the POMDP into a (continuous) belief-state MDP. At each step, b_t is updated via Bayesian filtering when a new observation arrives.

- (c) **Example — UAV tracking:** A pursuer UAV attempts to intercept an intruder drone.
 - *True state s_t :* The intruder's exact 3D position, velocity, and heading.
 - *Observation o_t :* Noisy radar returns (range, azimuth) and LiDAR point-cloud detections, both subject to measurement noise, limited field of view, and occasional missed detections.
 - *Source of partial observability:* Sensor noise, limited detection range, and occlusion mean the pursuer never knows the intruder's true state exactly and must maintain a belief over possible intruder states.

This is a bridge to Chapter 35: when the transition model is also unknown, the agent must *learn* under partial observability, motivating methods like SAC with observation fusion.

2 PART B: Mathematical Derivations

Problem B1: Bellman Equation Derivation

Starting from the definition of the state-value function

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right],$$

derive the Bellman expectation equation:

$$V^\pi(s) = \sum_a \pi(a|s) \left[R(s, a) + \gamma \sum_{s'} p(s'|s, a) V^\pi(s') \right].$$

Solution B1

Start from the recursive structure of the return $G_t = r_t + \gamma G_{t+1}$:

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi[G_0 \mid s_0 = s] \\ &= \mathbb{E}_\pi[r_0 + \gamma G_1 \mid s_0 = s] \\ &= \mathbb{E}_\pi[r_0 \mid s_0 = s] + \gamma \mathbb{E}_\pi[G_1 \mid s_0 = s]. \end{aligned} \quad (1)$$

First term: The immediate expected reward depends on the action chosen by π and the transition:

$$\mathbb{E}_\pi[r_0 \mid s_0 = s] = \sum_a \pi(a|s) R(s, a).$$

Second term: By the law of total expectation, conditioning on the action a_0 and next state s_1 :

$$\begin{aligned} \mathbb{E}_\pi[G_1 \mid s_0 = s] &= \sum_a \pi(a|s) \sum_{s'} p(s'|s, a) \mathbb{E}_\pi[G_1 \mid s_1 = s'] \\ &= \sum_a \pi(a|s) \sum_{s'} p(s'|s, a) V^\pi(s'). \end{aligned} \quad (2)$$

The last step uses the Markov property: $\mathbb{E}_\pi[G_1 \mid s_1 = s'] = V^\pi(s')$.

Combining (1) and (2):

$$V^\pi(s) = \sum_a \pi(a|s) \left[R(s, a) + \gamma \sum_{s'} p(s'|s, a) V^\pi(s') \right].$$

This is the **Bellman expectation equation**: the value of state s equals the expected immediate reward plus the discounted expected value of the next state, averaged over all actions and transitions.

Problem B2: Relationship Between V , Q , and A

Given the definitions:

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi[G_0 \mid s_0 = s], \\ Q^\pi(s, a) &= \mathbb{E}_\pi[G_0 \mid s_0 = s, a_0 = a], \\ A^\pi(s, a) &= Q^\pi(s, a) - V^\pi(s), \end{aligned}$$

prove that:

- (a) $V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s, a)$
- (b) $\mathbb{E}_{a \sim \pi(\cdot|s)}[A^\pi(s, a)] = 0$
- (c) $Q^\pi(s, a) = R(s, a) + \gamma \sum_{s'} p(s'|s, a) V^\pi(s')$

Solution B2

- (a) By the law of total expectation, marginalizing over the first action:

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi[G_0 \mid s_0 = s] \\ &= \sum_a \pi(a|s) \mathbb{E}_\pi[G_0 \mid s_0 = s, a_0 = a] \\ &= \sum_a \pi(a|s) Q^\pi(s, a). \quad \blacksquare \end{aligned}$$

(b) Using part (a) and the definition of advantage:

$$\begin{aligned}\mathbb{E}_{a \sim \pi}[A^\pi(s, a)] &= \sum_a \pi(a|s) [Q^\pi(s, a) - V^\pi(s)] \\ &= \sum_a \pi(a|s) Q^\pi(s, a) - V^\pi(s) \sum_a \pi(a|s) \\ &= V^\pi(s) - V^\pi(s) \cdot 1 = 0. \quad \blacksquare\end{aligned}$$

Interpretation: The advantage measures how much *better* action a is compared to the average action under π . By definition, the average advantage is zero.

(c) Starting from $Q^\pi(s, a) = \mathbb{E}_\pi[r_0 + \gamma G_1 \mid s_0 = s, a_0 = a]$:

$$\begin{aligned}Q^\pi(s, a) &= \mathbb{E}[r_0 \mid s, a] + \gamma \mathbb{E}_\pi[G_1 \mid s_0 = s, a_0 = a] \\ &= R(s, a) + \gamma \sum_{s'} p(s'|s, a) \mathbb{E}_\pi[G_1 \mid s_1 = s'] \\ &= R(s, a) + \gamma \sum_{s'} p(s'|s, a) V^\pi(s'). \quad \blacksquare\end{aligned}$$

Problem B3: Value Iteration Convergence

Show that Value Iteration is a contraction mapping. Specifically, let $\mathcal{T}$ denote the Bellman optimality operator:

$$(\mathcal{T}V)(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} p(s'|s, a) V(s') \right].$$

Prove that for any two value functions V_1, V_2 :

$$\|\mathcal{T}V_1 - \mathcal{T}V_2\|_\infty \leq \gamma \|V_1 - V_2\|_\infty,$$

where $\|V\|_\infty = \max_s |V(s)|$.

Solution B3

For any state s , let $a_1^* = \operatorname{argmax}_a [R(s, a) + \gamma \sum_{s'} p(s'|s, a) V_1(s')]$. Then:

$$(\mathcal{T}V_1)(s) - (\mathcal{T}V_2)(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} p(s'|s, a) V_1(s') \right] - \max_a \left[R(s, a) + \gamma \sum_{s'} p(s'|s, a) V_2(s') \right].$$

Using the inequality $\max_a f(a) - \max_a g(a) \leq \max_a [f(a) - g(a)]$:

$$\begin{aligned}(\mathcal{T}V_1)(s) - (\mathcal{T}V_2)(s) &\leq \max_a \left[\gamma \sum_{s'} p(s'|s, a) (V_1(s') - V_2(s')) \right] \\ &\leq \max_a \left[\gamma \sum_{s'} p(s'|s, a) \|V_1 - V_2\|_\infty \right] \\ &= \gamma \|V_1 - V_2\|_\infty.\end{aligned}$$

The last step uses $\sum_{s'} p(s'|s, a) = 1$. By symmetry (swapping V_1 and V_2), we also have $(\mathcal{T}V_2)(s) - (\mathcal{T}V_1)(s) \leq \gamma \|V_1 - V_2\|_\infty$. Therefore:

$$|(\mathcal{T}V_1)(s) - (\mathcal{T}V_2)(s)| \leq \gamma \|V_1 - V_2\|_\infty \quad \forall s.$$

Taking the max over s :

$$\|\mathcal{T}V_1 - \mathcal{T}V_2\|_\infty \leq \gamma \|V_1 - V_2\|_\infty.$$

Since $0 \leq \gamma < 1$, $\mathcal{T}$ is a contraction mapping. By the Banach fixed-point theorem, $\mathcal{T}$ has a unique fixed point V^* , and repeated application $V_{k+1} = \mathcal{T}V_k$ converges to V^* at a geometric rate γ^k .

3 PART C: Computational Problems

Problem C1: Computing Optimal Q-values

Consider the 1D grid world from the lecture with states $\{S_{T1}, S_1, S_2, S_3, S_{T2}\}$, where S_{T1} and S_{T2} are terminal (absorbing). The agent can move Up (toward S_{T1}) or Down (toward S_{T2}). Transitions are deterministic. Only $R(S_3, \text{Down}) = 1$ (reaching S_{T2}); all other rewards are 0.

Compute $Q^*(s, a)$ for all non-terminal state-action pairs using $\gamma = 0.9$.

Solution C1

Terminal states have $V^*(S_{T1}) = V^*(S_{T2}) = 0$. Using $Q^*(s, a) = R(s, a) + \gamma V^*(s')$ and $V^*(s) = \max_a Q^*(s, a)$:

Start from terminal-adjacent state S_3 :

$$\begin{aligned} Q^*(S_3, \downarrow) &= R(S_3, \downarrow) + \gamma V^*(S_{T2}) = 1 + 0 = 1.0 \\ Q^*(S_3, \uparrow) &= R(S_3, \uparrow) + \gamma V^*(S_2) = 0 + 0.9V^*(S_2). \end{aligned}$$

So $V^*(S_3) = \max(1.0, 0.9V^*(S_2))$. Clearly $V^*(S_3) = 1.0$ (going down is optimal from S_3).

State S_2 :

$$\begin{aligned} Q^*(S_2, \downarrow) &= 0 + 0.9 \times V^*(S_3) = 0.9 \times 1.0 = 0.9 \\ Q^*(S_2, \uparrow) &= 0 + 0.9 \times V^*(S_1) = 0.9V^*(S_1). \end{aligned}$$

Since $V^*(S_1) \leq 0.81 < 0.9$, we get $V^*(S_2) = 0.9$.

Now update $Q^*(S_3, \uparrow) = 0.9 \times 0.9 = 0.81$.

State S_1 :

$$\begin{aligned} Q^*(S_1, \uparrow) &= 0 + 0.9 \times V^*(S_{T1}) = 0 \\ Q^*(S_1, \downarrow) &= 0 + 0.9 \times V^*(S_2) = 0.9 \times 0.9 = 0.81. \end{aligned}$$

So $V^*(S_1) = 0.81$.

Check $Q^*(S_2, \uparrow) = 0.9 \times 0.81 = 0.729$.

Final Q^* table:

State	$Q^*(\cdot, \uparrow)$	$Q^*(\cdot, \downarrow)$
S_1	0	0.81
S_2	0.729	0.9
S_3	0.81	1.0

The optimal policy is $\pi^*(S_1) = \downarrow$, $\pi^*(S_2) = \downarrow$, $\pi^*(S_3) = \downarrow$ (always move toward S_{T2}).

Problem C2: Value Iteration by Hand

Consider an MDP with two non-terminal states $\{A, B\}$ and one terminal state T . The agent has two actions: a_1 and a_2 . The transition and reward model is:

State	Action	Next State	Prob.	Reward
A	a_1	B	1.0	0
A	a_2	T	1.0	2
B	a_1	A	0.5	0
B	a_1	T	0.5	10
B	a_2	A	1.0	1

Use $\gamma = 0.9$. Initialize $V_0(A) = V_0(B) = 0$. Perform **two iterations** of Value Iteration and determine the greedy policy after each iteration.

Solution C2

Value Iteration Update: $V_{k+1}(s) = \max_a [R(s, a) + \gamma \sum_{s'} p(s'|s, a) V_k(s')]$

Iteration 1 (using $V_0(A) = V_0(B) = 0$, $V(T) = 0$):

State A :

$$Q(A, a_1) = 0 + 0.9 \cdot V_0(B) = 0$$

$$Q(A, a_2) = 2 + 0.9 \cdot V_0(T) = 2$$

$\Rightarrow V_1(A) = \max(0, 2) = 2$. Greedy: a_2 .

State B :

$$Q(B, a_1) = 0.5[0 + 0.9 \cdot V_0(A)] + 0.5[10 + 0.9 \cdot V_0(T)] = 0 + 5 = 5$$

$$Q(B, a_2) = 1 + 0.9 \cdot V_0(A) = 1$$

$\Rightarrow V_1(B) = \max(5, 1) = 5$. Greedy: a_1 .

Iteration 2 (using $V_1(A) = 2$, $V_1(B) = 5$):

State A :

$$Q(A, a_1) = 0 + 0.9 \times 5 = 4.5$$

$$Q(A, a_2) = 2 + 0 = 2$$

$\Rightarrow V_2(A) = 4.5$. Greedy: **a_1** (changed!).

State B :

$$Q(B, a_1) = 0.5[0 + 0.9 \times 2] + 0.5[10 + 0] = 0.9 + 5 = 5.9$$

$$Q(B, a_2) = 1 + 0.9 \times 2 = 2.8$$

$\Rightarrow V_2(B) = 5.9$. Greedy: a_1 .

Summary:

	V_1	V_2	π_1	π_2
A	2.0	4.5	a_2	a_1
B	5.0	5.9	a_1	a_1

After iteration 2, the policy at A flipped from a_2 to a_1 . The agent now prefers to go to B first (for a chance at the large reward of 10) rather than going directly to T for reward 2.

Problem C3: UCB Action Selection

External Source

Adapted from Sutton & Barto, *Reinforcement Learning: An Introduction* (2nd ed.), Chapter 2 exercises on exploration strategies.

[EXT] A 3-armed bandit has the following estimated means and counts after 20 total pulls:

Arm	$\hat{\mu}(a)$	$N_t(a)$	$\hat{\mu} + c\sqrt{\frac{\ln t}{N_t(a)}}$
1	2.5	12	?
2	3.0	5	?
3	1.8	3	?

Using $c = 2$ and $t = 20$, compute the UCB score for each arm and determine which arm UCB selects. Discuss why UCB might not select the arm with the highest estimated mean.

Solution C3

We compute $\text{UCB}(a) = \hat{\mu}(a) + c\sqrt{\frac{\ln t}{N_t(a)}}$ with $c = 2$, $t = 20$, $\ln 20 \approx 3.0$:

Arm 1: $\text{UCB}(1) = 2.5 + 2\sqrt{\frac{3.0}{12}} = 2.5 + 2\sqrt{0.25} = 2.5 + 1.0 = 3.5$

Arm 2: $\text{UCB}(2) = 3.0 + 2\sqrt{\frac{3.0}{5}} = 3.0 + 2\sqrt{0.6} = 3.0 + 1.549 = 4.549$

Arm 3: $\text{UCB}(3) = 1.8 + 2\sqrt{\frac{3.0}{3}} = 1.8 + 2\sqrt{1.0} = 1.8 + 2.0 = 3.8$

UCB selects Arm 2 with score 4.549.

Although Arm 2 has the highest mean, even Arm 3 (lowest mean) beats Arm 1 in UCB score. This is because Arm 3 has been pulled only 3 times, giving it a large exploration bonus. UCB embodies “optimism in the face of uncertainty”: arms with fewer observations have wider confidence bounds, so the algorithm is optimistic about their true value and explores them more.

If Arm 3’s true mean turned out to be high, UCB would have found a valuable arm that pure exploitation (ϵ -greedy with small ϵ) might have missed.

4 PART D: Analysis & Comparison

Problem D1: Policy Iteration vs. Value Iteration

Compare and contrast Policy Iteration and Value Iteration. Specifically:

- Describe the two steps of Policy Iteration.
- What is the computational bottleneck of each method?
- Which method typically converges in fewer iterations? Which is faster per iteration?

Solution D1

(a) **Policy Iteration has two alternating steps:**

- Policy Evaluation:** Given fixed policy π , solve for V^π by solving the linear system $\mathbf{v} = \mathbf{r} + \gamma \mathbf{T} \mathbf{v}$, i.e., $\mathbf{v} = (\mathbf{I} - \gamma \mathbf{T})^{-1} \mathbf{r}$ where $T(s'|s) = \sum_a \pi(a|s) p(s'|s, a)$.
- Policy Improvement:** Make the policy greedy with respect to the current value function: $\pi'(s) = \operatorname{argmax}_a [R(s, a) + \gamma \sum_{s'} p(s'|s, a) V^\pi(s')]$.

(b) **Computational bottleneck:**

- *Policy Iteration:* The policy evaluation step requires solving a $|S| \times |S|$ linear system, costing $O(|S|^3)$ per iteration.
- *Value Iteration:* Each iteration performs a single Bellman backup over all states and actions, costing $O(|S|^2|A|)$ per iteration.

(c) **Convergence:**

- Policy Iteration converges in *fewer iterations* (often very few for small MDPs), guaranteed finite (at most $|A|^{|S|}$ policies).
- Value Iteration converges *asymptotically* at rate γ^k , but each iteration is much cheaper.
- In practice, for small state spaces, Policy Iteration is often faster overall. For large state spaces, Value Iteration (or modified PI with truncated evaluation) is preferred.

Part II

Chapter 35: Reinforcement Learning

5 PART A: Conceptual Questions

Problem A1: Planning vs. Learning

What is the fundamental difference between planning (Chapter 34) and reinforcement learning (Chapter 35)? What information does the RL agent lack compared to a planning agent?

Solution A1

Planning (Ch. 34): The agent has access to a *complete model* of the environment, i.e., the transition probabilities $p(s'|s, a)$ and the reward function $R(s, a)$ are known. The agent can compute the optimal policy π^* *without ever interacting* with the environment, using methods like Value Iteration or Policy Iteration.

Reinforcement Learning (Ch. 35): The model is *unknown*. The agent must learn by *interacting* with the environment and observing (s, a, r, s') tuples. It discovers the consequences of its actions through trial and error.

What RL lacks:

- The transition model $p(s'|s, a)$: the agent doesn't know where its actions will lead.
- The reward function $R(s, a)$: the agent doesn't know which states/actions are rewarding until it tries them.

This makes RL fundamentally harder because the agent must balance exploration (learning the model/rewards) with exploitation (using what it has learned to maximize reward).

Problem A2: Monte Carlo vs. TD Learning

Compare Monte Carlo (MC) and Temporal Difference (TD) methods along the following dimensions:

- (a) When can the value estimate be updated?
- (b) Bias–variance tradeoff
- (c) Which requires episodes to terminate?

Solution A2

(a) Update timing:

- *MC*: Must wait until the *end of the episode* to compute the full return $G_t = r_t + \gamma r_{t+1} + \dots + \gamma^{T-t-1} r_{T-1}$, then update.
- *TD(0)*: Updates after *every single step* using the bootstrapped target $r_t + \gamma V(s_{t+1})$.

(b) Bias–variance:

- *MC*: **Unbiased** estimate of Q^π (uses the true return). **High variance** because G_t is a sum of many random variables.
- *TD(0)*: **Biased** because it bootstraps from $V(s_{t+1})$, which is itself an estimate. **Lower variance** because only one random step is involved.

(c) **Episode requirement:**

- *MC*: Requires episodes to terminate (cannot be used in continuing tasks).
- *TD*: Works for both episodic and continuing tasks.

Analogy from the lecture: TD learning is like updating your ETA to Toronto once you reach Kingston (one step), while MC is like only updating your estimate after you actually arrive in Toronto (full episode).

Problem A3: On-Policy vs. Off-Policy

- (a) Define on-policy and off-policy learning.
- (b) Classify each of the following methods: SARSA, Q-learning, REINFORCE, PPO, DQN, SAC.
- (c) Why is off-policy learning important in practice?

Solution A3

- (a) **On-policy:** The agent learns about the policy π that it is currently using to generate data. Data must come from the current policy; old data is discarded when π changes.

Off-policy: The agent learns about a target policy π using data generated by a different behavior policy π_b . Old data can be reused via a replay buffer.

- (b) Classification:

On-policy	Off-policy
SARSA	Q-learning
REINFORCE	DQN
PPO	SAC

- (c) **Practical importance:** Off-policy methods are essential when:

- Real-world interaction is expensive (robotics, clinical trials, UAVs).
- Data collection is slow—reusing past experience via a replay buffer is much more sample-efficient.
- We want to learn from demonstrations or historical data (e.g., learning from human expert trajectories).

The tradeoff is that off-policy methods can be unstable due to the “deadly triad” (off-policy + function approximation + bootstrapping).

Problem A4: The Deadly Triad

What is the “deadly triad” in reinforcement learning? Why does combining all three elements lead to instability?

Solution A4

The **deadly triad** (Sutton & Barto) refers to the combination of three elements that can cause Q-value estimates to diverge:

1. **Off-policy learning:** Learning about a policy different from the one generating data.
2. **Function approximation:** Using a neural network (or other function approximator) instead of a table to represent V or Q .
3. **Bootstrapping:** Using the current value estimate (e.g., $V(s')$) as part of the update target.

Why the combination is dangerous: Each element alone is manageable. However, when combined:

- Off-policy data can have a different distribution than what the function approximator was trained on, causing errors in regions of state space that are visited under the behavior policy but not the target policy.
- Bootstrapping propagates these approximation errors: if $Q(s', a')$ is overestimated, this error feeds back into the update for $Q(s, a)$.
- Function approximation generalizes errors across similar states, creating a positive feedback loop where overestimation grows without bound.

Mitigations: Target networks (DQN), twin critics (TD3, SAC), gradient clipping, and careful replay buffer management all address aspects of this instability.

Problem A5: DQN Innovations

Describe the two key innovations introduced by DQN (Mnih et al., 2015) that enabled stable training of deep Q-networks. Why is each necessary?

Solution A5

1. Experience Replay:

- Store transitions (s, a, r, s') in a replay buffer $\mathcal{D}$.
- Sample random mini-batches from $\mathcal{D}$ for training.
- *Why necessary:* Consecutive transitions are highly correlated (e.g., consecutive video frames). Training a neural network on correlated data violates the i.i.d. assumption of SGD and leads to poor convergence. Random sampling breaks this temporal correlation. Additionally, reusing past experience improves sample efficiency.

2. Target Network:

- Maintain a separate “frozen” copy of the Q-network Q_{w^-} to compute the target $y = r + \gamma \max_{a'} Q_{w^-}(s', a')$.
- The target network parameters w^- are periodically synced with the main network w (e.g., every C steps).
- *Why necessary:* Without a target network, the target $r + \gamma \max_{a'} Q_w(s', a')$ changes with every gradient step on w . This creates a “moving target” problem—like trying to chase a goal that keeps shifting. Freezing the target network provides a stable regression target for several steps, preventing oscillations and divergence.

Together, these innovations address two of the three elements of the deadly triad: experience replay decorrelates data (mitigating off-policy issues), and target networks stabilize bootstrapping.

6 PART B: Mathematical Derivations

Problem B1: SARSA vs. Q-Learning Update Derivation

Write out the full update rules for SARSA and Q-learning. Starting from the general TD update pattern, show how the two differ in their choice of “next action” and explain what each converges to.

Solution B1

Both SARSA and Q-learning are TD control methods with the general form:

$$Q(s, a) \leftarrow Q(s, a) + \eta [\text{target} - Q(s, a)].$$

SARSA (on-policy): The agent observes the tuple (s, a, r, s', a') where $a' \sim \pi(s')$ is the action *actually taken* next:

$$Q(s, a) \leftarrow Q(s, a) + \eta [r + \gamma Q(s', a') - Q(s, a)]$$

where a' is sampled from the current (e.g., ϵ -greedy) policy.

SARSA converges to Q^π (the Q-function of the *current* policy π). With GLIE conditions ($\epsilon \rightarrow 0$), it converges to Q^* .

Q-learning (off-policy): Replace the sampled a' with the greedy maximum:

$$Q(s, a) \leftarrow Q(s, a) + \eta \left[r + \gamma \max_b Q(s', b) - Q(s, a) \right]$$

Q-learning directly targets Q^* regardless of the behavior policy, provided standard convergence conditions hold (such as sufficient exploration and appropriate learning-rate conditions).

Key difference:

	SARSA	Q-learning
Target	$r + \gamma Q(s', a')$	$r + \gamma \max_b Q(s', b)$
Next action	$a' \sim \pi(s')$	$\operatorname{argmax}_b Q(s', b)$
Learns	Q^π	Q^*
Behavior	Cautious	Aggressive

In the cliff-walking problem, SARSA learns a safe path away from the cliff (because the ϵ -greedy behavior occasionally falls off), while Q-learning learns the optimal path along the cliff edge (it assumes perfect future behavior).

Problem B2: Policy Gradient Theorem

State the policy gradient theorem and show intuitively how it leads to the REINFORCE update. Given a parameterized policy $\pi_\theta(a|s)$, we define the objective $J(\pi_\theta) = \mathbb{E}_\pi[G_0]$.

Derive the gradient $\nabla_\theta J(\pi_\theta)$ and explain the role of the baseline $b(s)$.

Solution B2

Policy Gradient Theorem:

$$\nabla_{\theta} J(\pi_{\theta}) \propto \mathbb{E}_{\pi} [\nabla_{\theta} \log \pi_{\theta}(a|s) \cdot Q^{\pi_{\theta}}(s, a)].$$

REINFORCE update: In practice, the policy gradient theorem yields an expectation over state-action visitation under the current policy, and $Q^{\pi}(s, a)$ is commonly replaced by the sampled return G_t :

$$\theta \leftarrow \theta + \eta \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot G_t.$$

Intuition:

- $\nabla_{\theta} \log \pi_{\theta}(a|s)$: the direction in parameter space that *increases* the probability of action a .
- $Q^{\pi}(s, a)$ (or G_t): how “good” that action turned out to be.
- If $Q > 0$: move θ to make a more likely.
- If $Q < 0$: move θ to make a less likely.

Baseline $b(s)$: We can subtract any function of s alone without changing the expectation:

$$\nabla_{\theta} J = \mathbb{E}_{\pi} [\nabla_{\theta} \log \pi_{\theta}(a|s) \cdot (Q^{\pi}(s, a) - b(s))].$$

This holds because $\mathbb{E}_{a \sim \pi} [\nabla_{\theta} \log \pi_{\theta}(a|s)] = \nabla_{\theta} \sum_a \pi_{\theta}(a|s) = \nabla_{\theta} 1 = 0$.

A common choice is $b(s) = V^{\pi}(s)$, giving the advantage $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$.

Why a baseline helps: Without a baseline, all observed returns might be positive (e.g., a game where you always score > 0). Then *all* actions get reinforced, just with different magnitudes. The baseline centers the signal around zero so that below-average actions are explicitly suppressed, dramatically reducing variance.

Problem B3: Double Q-Learning

Explain the maximization bias in standard Q-learning and derive the Double Q-learning update rule. Why does using two Q-functions solve the problem?

Solution B3

Maximization Bias: In Q-learning, the target uses $\max_b Q(s', b)$. If Q-values have estimation noise, the max operator systematically *overestimates* the true value because:

$$\mathbb{E} \left[\max_b Q(s', b) \right] \geq \max_b \mathbb{E}[Q(s', b)].$$

This is Jensen’s inequality applied to the convex max function. The overestimation creates a positive feedback loop: overestimated Q-values lead to preferring those actions, which further inflates their estimates.

Double Q-Learning Update: Maintain two independent Q-functions Q_1 and Q_2 . With probability 0.5, update Q_1 using Q_2 for evaluation:

$$\boxed{Q_1(s, a) \leftarrow Q_1(s, a) + \eta \left[r + \gamma Q_2 \left(s', \operatorname{argmax}_{a'} Q_1(s', a') \right) - Q_1(s, a) \right]}$$

(and symmetrically for Q_2).

Why this works: The key idea is *decoupling selection from evaluation*:

- Q_1 **selects** (nominates) the best action: $a^* = \operatorname{argmax}_{a'} Q_1(s', a')$.
- Q_2 **evaluates** that action: $Q_2(s', a^*)$.

Since Q_1 and Q_2 have independent estimation noise, even if Q_1 overestimates an action, Q_2 is unlikely to overestimate the same action. This breaks the positive feedback loop.

Legacy: This principle extends to target networks in DQN and twin critics in TD3/SAC.

7 PART C: Computational Problems

Problem C1: Q-Learning Step-by-Step

External Source

Adapted from Sutton & Barto, *Reinforcement Learning: An Introduction* (2nd ed.), Example 6.6 (Cliff Walking concept) and Ch. 6 exercises.

[EXT] Consider the 1D grid world from the lecture. Apply Q-learning with $\gamma = 0.9$, $\eta = 1.0$ (full replacement), and initialize all $Q(s, a) = 0$.

The agent follows an ε -greedy policy with $\varepsilon = 1.0$ (fully random) and experiences the following episodes:

Episode 1: Start at S_3 , action $\downarrow$, reward 1, reach S_{T2} .

Episode 2: Start at S_2 , action $\downarrow$, reward 0, reach S_3 ; then action $\downarrow$, reward 1, reach S_{T2} .

Episode 3: Start at S_1 , action $\downarrow$, reward 0, reach S_2 ; action $\uparrow$, reward 0, reach S_1 ; action $\downarrow$, reward 0, reach S_2 ; action $\downarrow$, reward 0, reach S_3 ; action $\downarrow$, reward 1, reach S_{T2} .

Trace the Q-table after each transition.

Solution C1

With $\eta = 1$, the Q-learning update $Q(s, a) \leftarrow r + \gamma \max_b Q(s', b)$ is a full replacement.

Episode 1:

- $(S_3, \downarrow, 1, S_{T2})$: $Q(S_3, \downarrow) \leftarrow 1 + 0.9 \times 0 = 1.0$

Episode 2:

- $(S_2, \downarrow, 0, S_3)$: $Q(S_2, \downarrow) \leftarrow 0 + 0.9 \times \max\{Q(S_3, \uparrow), Q(S_3, \downarrow)\} = 0.9 \times 1.0 = 0.9$
- $(S_3, \downarrow, 1, S_{T2})$: $Q(S_3, \downarrow) \leftarrow 1 + 0 = 1.0$ (unchanged)

Episode 3:

- $(S_1, \downarrow, 0, S_2)$: $Q(S_1, \downarrow) \leftarrow 0 + 0.9 \times \max\{0, 0.9\} = 0.81$
- $(S_2, \uparrow, 0, S_1)$: $Q(S_2, \uparrow) \leftarrow 0 + 0.9 \times \max\{0, 0.81\} = 0.729$
- $(S_1, \downarrow, 0, S_2)$: $Q(S_1, \downarrow) \leftarrow 0 + 0.9 \times \max\{0.729, 0.9\} = 0.81$ (unchanged)
- $(S_2, \downarrow, 0, S_3)$: $Q(S_2, \downarrow) \leftarrow 0 + 0.9 \times 1.0 = 0.9$ (unchanged)
- $(S_3, \downarrow, 1, S_{T2})$: $Q(S_3, \downarrow) \leftarrow 1.0$ (unchanged)

Final Q-table after 3 episodes:

State	$Q(\cdot, \uparrow)$	$Q(\cdot, \downarrow)$
S_1	0	0.81
S_2	0.729	0.9
S_3	0	1.0

This already matches Q^* ! With $\eta = 1$ (full replacement) and sufficient exploration, Q-learning converges quickly. Note that $Q(S_3, \uparrow)$ and $Q(S_1, \uparrow)$ remain 0 because those state-action pairs were never visited with rewards that would make them nonzero through the “up” direction.

Problem C2: SARSA vs. Q-Learning Comparison

Consider a simple 3-state MDP with states $\{A, B, \text{Cliff}\}$ where B is the goal (reward +10) and Cliff gives reward -100 . From state A , action “safe” goes to B with probability 1, and action “risky” goes to B with probability 0.8 and to Cliff with probability 0.2.

Using $\gamma = 1$ (episodic), compute:

- (a) The Q-values that Q-learning converges to.
- (b) The Q-values that SARSA converges to under a fixed ε -greedy policy with $\varepsilon = 0.1$.
- (c) Which algorithm recommends the “risky” action? Is this appropriate?

Solution C2

Both B and Cliff are terminal. $\gamma = 1$.

(a) Q-learning (learns Q^*):

Q-learning uses $\max_b Q(s', b)$ in the target, evaluating the *optimal* future policy:

$$\begin{aligned} Q^*(A, \text{safe}) &= 1.0 \times 10 = 10 \\ Q^*(A, \text{risky}) &= 0.8 \times 10 + 0.2 \times (-100) = 8 - 20 = -12 \end{aligned}$$

Q-learning recommends “safe” ($Q^* = 10 > -12$).

(b) SARSA with $\varepsilon = 0.1$ (learns Q^π):

Under ε -greedy with $\varepsilon = 0.1$, the agent follows the greedy action 95% of the time and random 5% per action (2 actions, so each random action has probability 0.05).

SARSA’s target uses the action $a' \sim \pi_\varepsilon(s')$. Since B and Cliff are terminal, the values from the next state are 0. But crucially, SARSA evaluates the *current policy*, which includes the exploratory actions.

For state A under ε -greedy where “safe” is greedy:

$$\begin{aligned} Q^\pi(A, \text{safe}) &= 10 \\ Q^\pi(A, \text{risky}) &= 0.8 \times 10 + 0.2 \times (-100) = -12 \end{aligned}$$

In this simple terminal-reward case, the Q-values are the same. But in a multi-step cliff-walking scenario, SARSA would account for the fact that ε -greedy exploration near the cliff leads to occasional falls, making paths near the cliff have lower value.

(c) Both algorithms recommend “safe” here. In general, Q-learning finds the theoretically optimal policy (assuming perfect execution), while SARSA finds the best policy *given that the agent will continue to explore*. SARSA is more appropriate when ε remains fixed during deployment.

Problem C3: TD Error Computation

External Source

Inspired by Sutton & Barto, *Reinforcement Learning: An Introduction* (2nd ed.), Exercise 6.1.

[**EXT**] An agent using TD(0) with $\eta = 0.1$ has the following value estimates:

$$V(A) = 5.0, \quad V(B) = 8.0, \quad V(C) = 3.0.$$

The agent is in state A , takes an action, receives reward $r = 2$, and transitions to state B . Using $\gamma = 0.9$:

- (a) Compute the TD error δ .
- (b) Compute the updated value $V(A)$.
- (c) If the true value $V^*(A) = 10.0$, is the update moving in the right direction?

Solution C3

- (a) **TD error:**

$$\delta = r + \gamma V(B) - V(A) = 2 + 0.9 \times 8.0 - 5.0 = 2 + 7.2 - 5.0 = 4.2.$$

The positive TD error means the observed outcome ($r + \gamma V(B) = 9.2$) was *better* than expected ($V(A) = 5.0$).

- (b) **Updated value:**

$$V(A) \leftarrow V(A) + \eta \cdot \delta = 5.0 + 0.1 \times 4.2 = 5.0 + 0.42 = 5.42.$$

- (c) **Direction check:** Since $V^*(A) = 10.0$ and we updated from 5.0 to 5.42, the update is indeed moving $V(A)$ *toward* the true value. The TD error correctly identified that the current estimate was too low. Over many updates, $V(A)$ will converge toward $V^*(A)$.

8 PART D: Analysis & Comparison

Problem D1: Actor-Critic Architecture

Explain the Actor-Critic architecture. What does each component learn, and how do they interact? Why is this considered “the best of both worlds”?

Solution D1

The Actor-Critic architecture has two components:

Actor $\pi_\theta(a|s)$: A parameterized policy that maps states to action probabilities. The actor decides *what to do*.

Critic $V_w(s)$ (or $Q_w(s, a)$): A value function estimator that evaluates how good the current state (or action) is. The critic provides *feedback*.

Interaction:

1. The actor takes action a_t in state s_t .
2. The environment returns reward r_t and next state s_{t+1} .
3. The critic computes the TD error (“surprise score”):

$$\delta_t = r_t + \gamma V_w(s_{t+1}) - V_w(s_t).$$

4. The actor updates θ using δ_t as a learning signal:

$$\theta \leftarrow \theta + \eta_\theta \delta_t \nabla_\theta \log \pi_\theta(a_t|s_t).$$

5. The critic updates w :

$$w \leftarrow w + \eta_w \delta_t \nabla_w V_w(s_t).$$

“Best of both worlds”:

- From *policy methods*: handles continuous action spaces naturally, can learn stochastic policies.
- From *value methods*: step-by-step learning (no need to wait for episode end), lower variance than pure policy gradient (REINFORCE) because the critic provides per-step advantage estimates instead of noisy full-episode returns.

Analogy from the lecture: The actor is a comedian testing jokes, and the critic is the audience providing immediate feedback after each joke, rather than waiting until the end of the show.

Problem D2: PPO Clipping Mechanism

Explain how PPO’s clipped objective prevents destructive large policy updates. Given the PPO objective:

$$J^{\text{PPO}}(\pi') = \mathbb{E} \left[\min \left(\frac{\pi'(a|s)}{\pi(a|s)} A^\pi, \text{clip} \left(\frac{\pi'(a|s)}{\pi(a|s)}, 1 - \varepsilon, 1 + \varepsilon \right) A^\pi \right) \right],$$

analyze what happens for (a) a good action ($A^\pi > 0$) and (b) a bad action ($A^\pi < 0$) when the ratio $r(\theta) = \pi'(a|s)/\pi(a|s)$ grows large.

Solution D2

Let $r(\theta) = \frac{\pi'(a|s)}{\pi(a|s)}$ be the importance sampling ratio.

(a) Good action ($A^\pi > 0$):

The unclipped objective $r(\theta)A^\pi$ increases without bound as $r(\theta) \rightarrow \infty$ (making the action much more probable). The clipped version caps $r(\theta)$ at $1 + \varepsilon$:

$$J = \min(r(\theta) A^\pi, (1 + \varepsilon) A^\pi).$$

When $r(\theta) > 1 + \varepsilon$, the clipped surrogate stops rewarding further increases in that direction, so the objective no longer encourages an arbitrarily large update. This means: “yes, this action is good, but don’t increase its probability by more than a factor of $1 + \varepsilon$.”

(b) Bad action ($A^\pi < 0$):

The unclipped objective $r(\theta)A^\pi$ improves (becomes less negative) as $r(\theta) \rightarrow 0$ (reducing the action’s probability). The clip caps at $1 - \varepsilon$:

$$J = \min(r(\theta) A^\pi, (1 - \varepsilon) A^\pi).$$

Since $A^\pi < 0$, $\min$ selects the *more negative* value. When $r(\theta) < 1 - \varepsilon$, the clipped surrogate similarly prevents the objective from encouraging an excessively large decrease in probability.

Summary: PPO limits the policy change per update to a “trust region” of $[1 - \varepsilon, 1 + \varepsilon]$ around the old policy. Typical values are $\varepsilon = 0.1$ or 0.2 . This achieves the stability of trust-region methods (like TRPO) without the computational cost of explicitly computing KL divergence constraints.

Problem D3: SAC Entropy Bonus

Soft Actor-Critic (SAC) augments the standard RL objective with an entropy term:

$$J^{\text{SAC}}(\theta) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t (R(s_t, a_t) + \lambda \mathcal{H}(\pi_{\theta}(\cdot|s_t))) \right].$$

- (a) What does the entropy term $\mathcal{H}(\pi_{\theta}(\cdot|s))$ encourage?
- (b) What happens as $\lambda \rightarrow 0$?
- (c) Why does SAC use twin critics?

Solution D3

- (a) Since SAC operates in continuous action spaces with a Gaussian policy $\pi_{\theta}(a|s) = \mathcal{N}(\mu_{\theta}(s), \sigma_{\theta}(s))$, the entropy is the *differential entropy*:

$$\mathcal{H}(\pi_{\theta}(\cdot|s)) = - \int \pi_{\theta}(a|s) \log \pi_{\theta}(a|s) da.$$

For a Gaussian with diagonal covariance, this equals $\frac{1}{2} \sum_{i=1}^d \log(2\pi e \sigma_i^2(s))$, which grows with the standard deviation. Maximizing entropy encourages the policy to:

- Remain *stochastic* and explore diverse actions.
- Avoid collapsing to a deterministic policy prematurely.
- Be robust: among policies with similar expected return, prefer the one that commits least to specific actions (maximum entropy principle).

Early in training, the entropy bonus promotes broad exploration. As the policy improves, λ can be automatically adjusted (decreased) to allow more exploitation.

- (b) As $\lambda \rightarrow 0$, the entropy term vanishes and SAC reduces to standard (non-entropy-regularized) RL. The policy becomes purely reward-maximizing and may become deterministic.
- (c) SAC uses **twin critics** Q_{w_1} and Q_{w_2} and takes $\min(Q_{w_1}, Q_{w_2})$ as the value estimate. This addresses *maximization bias* (the same problem as in Double Q-learning): a single critic tends to overestimate Q-values, which the actor then exploits. Taking the minimum of two independent estimates is pessimistic, which counteracts overestimation and leads to more stable training.

Problem D4: Case Study — Algorithm Selection for UAV Pursuit-Evasion

Consider a UAV pursuit-evasion scenario: an autonomous pursuer drone must intercept an evading intruder in 3D airspace. The pursuer outputs continuous thrust and heading commands, has access only to noisy radar and LiDAR observations, and each real-world flight is expensive.

Argue why SAC is a more suitable choice than PPO or DQN for this problem. Structure your answer around three axes:

- (a) Action space
- (b) Sample efficiency
- (c) Partial observability and exploration

Solution D4

- (a) **Action space:** The pursuer must output continuous control commands (thrust magnitude, heading angles). DQN requires a finite action set and computing $\max_a Q(s, a)$ over all actions; discretizing a multi-dimensional continuous space leads to an exponential explosion of actions and loss of fine-grained control. SAC and PPO both handle continuous actions natively via parameterized Gaussian policies.
- (b) **Sample efficiency:** Each training episode involves running a full flight simulation (and eventually real hardware), making data expensive. PPO is on-policy: it must discard all collected experience after every policy update and gather fresh rollouts. SAC is off-policy and stores all past transitions in a replay buffer, reusing each experience many times. This makes SAC significantly more sample-efficient—a critical advantage when simulation is slow or hardware is involved.
- (c) **Partial observability and exploration:** The pursuer receives noisy, incomplete sensor readings rather than the true intruder state, creating a POMDP-like setting. SAC's entropy-regularized objective $J = \mathbb{E}[R + \lambda \mathcal{H}(\pi)]$ maintains a stochastic policy that naturally explores diverse pursuit strategies, preventing premature convergence to a single approach that may fail when observations are ambiguous. The automatic temperature tuning reduces exploration as the policy matures. By contrast, PPO's exploration relies solely on the current policy's stochasticity (which can vanish), and DQN's ε -greedy exploration is state-independent and poorly suited to continuous spaces.

Summary: SAC combines continuous-action support, off-policy sample efficiency, and principled entropy-driven exploration—all essential properties for the UAV pursuit-evasion setting. This is why the lecture's “When to Use What” table recommends SAC (with domain-specific extensions such as fuzzy inference) for this class of problems.

9 PART E: External Resource Problems

Problem E1: Bellman Equation for q_* [EXT]

External Source

Adapted from Sutton & Barto, *Reinforcement Learning: An Introduction* (2nd ed.), Exercise 3.20.

Give an equation for q_* in terms of v_* and the four-argument dynamics function $p(s', r | s, a)$.

Solution E1

The optimal action-value function can be expressed as:

$$q_*(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')].$$

This says: the value of taking action a in state s under the optimal policy equals the expected immediate reward plus the discounted optimal value of the next state, where the expectation is over all possible next states s' and rewards r .

Equivalently, using $R(s, a) = \sum_{s', r} p(s', r | s, a) r$ and $p(s' | s, a) = \sum_r p(s', r | s, a)$:

$$q_*(s, a) = R(s, a) + \gamma \sum_{s'} p(s' | s, a) v_*(s').$$

To express v_* in terms of q_* :

$$v_*(s) = \max_a q_*(s, a).$$

Substituting back gives the Bellman optimality equation for q_* :

$$q_*(s, a) = \sum_{s', r} p(s', r | s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right].$$

Problem E2: Why Does TD(0) Have Lower Variance? [EXT]

External Source

Adapted from Sutton & Barto, *Reinforcement Learning: An Introduction* (2nd ed.), Chapter 6 discussion, and Murphy, *Probabilistic Machine Learning: Advanced Topics*, Section 35.2.

The MC target is $G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$. The TD(0) target is $r_t + \gamma V(s_{t+1})$. Explain mathematically why TD(0) has lower variance.

Solution E2

The variance of the MC target involves the sum of all future rewards:

$$\text{Var}(G_t) = \text{Var}\left(\sum_{k=0}^{T-t-1} \gamma^k r_{t+k}\right).$$

Even if rewards are independent, $\text{Var}(G_t) = \sum_{k=0}^{T-t-1} \gamma^{2k} \text{Var}(r_{t+k})$, which grows with the episode length.

The TD(0) target only involves *one* random reward:

$$\text{Var}(r_t + \gamma V(s_{t+1})) = \text{Var}(r_t) + \gamma^2 \text{Var}(V(s_{t+1})) + 2\gamma \text{Cov}(r_t, V(s_{t+1})).$$

If V is a good approximation of V^π , then $V(s_{t+1}) \approx \mathbb{E}[G_{t+1} | s_{t+1}]$ has much lower variance than G_{t+1} itself, since the expectation smooths out future randomness.

Key tradeoff:

- MC: high variance (sum of many random variables), zero bias.
- TD(0): lower variance (one random step + a smooth estimate), but biased if V is inaccurate.

As the value estimate improves over training, the bias introduced by bootstrapping typically decreases, while TD(0) retains its lower-variance target.

Problem E3: Exploration in Continuous Spaces [EXT]

External Source

Adapted from Lillicrap et al., *Continuous Control with Deep Reinforcement Learning* (DDPG, 2016) and Haarnoja et al., *Soft Actor-Critic* (2018).

DDPG uses a deterministic policy $a = \mu_\theta(s)$. How does it explore? Compare this to SAC's approach to exploration in continuous action spaces.

Solution E3

DDPG exploration: Since the policy is deterministic, DDPG adds *external noise* to actions during training:

$$a_t = \mu_\theta(s_t) + \mathcal{N}_t,$$

where $\mathcal{N}_t$ is typically Ornstein–Uhlenbeck noise (temporally correlated) or simple Gaussian noise. The noise scale is a hyperparameter that must be manually tuned and typically annealed during training.

Problems with DDPG’s approach:

- Noise is added in action space, not adapted to the task or state.
- The amount of exploration is controlled by a fixed schedule, not learned.
- A single critic can overestimate Q-values, causing the actor to exploit errors.

SAC exploration: SAC learns a *stochastic* policy $\pi_\theta(a|s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta(s))$ and maximizes an entropy-augmented objective. Exploration is built into the policy itself:

- The entropy bonus $\lambda \mathcal{H}(\pi)$ encourages the policy to maintain spread.
- The temperature λ is *automatically tuned* by constraining entropy to a target value.
- Early in training, high entropy drives broad exploration; as the agent learns, entropy naturally decreases.

Key advantage of SAC: Exploration is principled, adaptive, and state-dependent (the policy can be more uncertain in unfamiliar states and more deterministic in well-understood states), whereas DDPG’s noise is blind and uniform.

Problem E4: When to Use What? [EXT]

External Source

Based on the method selection table from Murphy, *Probabilistic Machine Learning: Advanced Topics*, Table 35.1, and the lecture summary slide.

For each of the following scenarios, recommend an RL algorithm and justify your choice:

- Atari games (discrete actions, pixel observations)
- Robotic arm control (continuous actions, limited real-world data)
- Fine-tuning a large language model from human feedback
- A board game with a known simulator (e.g., Go)

Solution E4

- Atari** → **DQN / Rainbow**. Discrete actions make $\max_a Q(s, a)$ tractable. CNNs process pixel inputs. Experience replay and target networks provide stability. Rainbow combines Double DQN, prioritized replay, dueling architecture, and more for state-of-the-art Atari performance.

- (b) **Robotic arm** $\rightarrow$ **SAC**. Continuous action space eliminates DQN. Off-policy learning is critical because real-world robot interactions are expensive—SAC’s replay buffer maximizes sample efficiency. Entropy regularization provides robust exploration and prevents premature convergence. Twin critics prevent overestimation.
- (c) **LLM fine-tuning (RLHF)** $\rightarrow$ **PPO**. PPO is on-policy and stable, which is important when the reward model (human feedback) is noisy and expensive. The clipping mechanism prevents catastrophic policy changes that could ruin language quality. PPO is the standard choice for InstructGPT-style training.
- (d) **Board games with simulator** $\rightarrow$ **MCTS + neural networks (AlphaZero)**. A perfect simulator enables model-based planning. Monte Carlo Tree Search explores the game tree efficiently, guided by a neural network that evaluates positions and suggests moves. Self-play generates unlimited training data.

Problem E5: ε -Greedy vs. Boltzmann Exploration [EXT]

External Source

Based on Murphy, *Probabilistic Machine Learning: Advanced Topics*, Table 35.2.

Given two actions with estimated rewards $\hat{R}(s, a_1) = 4.9$ and $\hat{R}(s, a_2) = 5.1$:

- (a) Compute the selection probabilities under ε -greedy with $\varepsilon = 0.1$.
- (b) Compute the selection probabilities under Boltzmann exploration with $\tau = 1$.
- (c) Which method is more robust to estimation noise and why?

Solution E5

- (a) **ε -greedy** ($\varepsilon = 0.1$): Since $\hat{R}(a_2) > \hat{R}(a_1)$, a_2 is greedy:

$$\begin{aligned}\pi_\varepsilon(a_1) &= \varepsilon/2 = 0.05 \\ \pi_\varepsilon(a_2) &= 1 - \varepsilon + \varepsilon/2 = 0.95\end{aligned}$$

- (b) **Boltzmann** ($\tau = 1$):

$$\begin{aligned}\pi_\tau(a_1) &= \frac{e^{4.9/1}}{e^{4.9/1} + e^{5.1/1}} = \frac{e^{4.9}}{e^{4.9} + e^{5.1}} = \frac{1}{1 + e^{0.2}} = \frac{1}{1 + 1.221} = 0.45 \\ \pi_\tau(a_2) &= 1 - 0.45 = 0.55\end{aligned}$$

- (c) **Boltzmann is more robust.** The Q-values differ by only 0.2, which is well within typical estimation noise. Yet ε -greedy assigns 95% probability to a_2 —a sharp jump that would flip entirely if the estimates were reversed by a tiny perturbation. Boltzmann assigns 55%/45%, reflecting that the evidence for a_2 being better is weak. When estimates are close, Boltzmann smoothly shares probability; when they are far apart, it becomes nearly greedy. ε -greedy always makes a hard jump at the boundary.

Summary of External Sources

- **Sutton & Barto**, *Reinforcement Learning: An Introduction*, 2nd edition (2018): Problems C1, C3 (Part II), E1, E2
- **Murphy**, *Probabilistic Machine Learning: Advanced Topics* (2023): Problems E4, E5
- **Lillicrap et al.** (DDPG, 2016) and **Haarnoja et al.** (SAC, 2018): Problem E3

Acknowledgements / Document Preparation

The problems and solutions in this document were generated with assistance from large language models (ChatGPT by OpenAI, CoPilot by Microsoft, and Claude by Anthropic), based on the course slide and notebook material. The instructor reviewed, modified, and verified all questions, solutions, and final formatting.